

TP FINAL – Módulo 2 – Programación Orientada a Objetos

Programación II 2025

Pautas de resolución y entrega

- Lean detenidamente el enunciado. Desarrollen la solución con su compañero/a en BlueJ. → Porfa
- Para la entrega, deben exportar el proyecto con el nombre **grupoXX.zip** (dónde XX es el número de grupo asignado) y uno/a de los integrantes debe subirlo en la tarea correspondiente en el entorno Asignaturas.

Enunciado

Una veterinaria necesita procesar la información de las atenciones realizadas a sus animales en el transcurso del año 2025. Para esto, la veterinaria posee un conjunto de atenciones (a lo sumo 30). De cada atención se conoce: número, fecha de la atención (con formato "DD/MM/AAAA"), tipo de animal (gato, perro, etc.), si el animal tiene el calendario de vacunación o no, precio, y la información correspondiente al dueño. De cada dueño se registra: nombre, apellido, DNI y método de pago de la atención (débito, mercado pago, crédito o efectivo).

Existen diferentes tipos de atenciones:

- **Tratamiento**, es aquella atención que registra la descripción del tratamiento y la sintomatología (cantidad de síntomas).
- **Intervención Quirúrgica**, es aquella atención que registra la duración (en minutos), si es programada o no, y el conjunto de nombres de los medicamentos utilizados (a lo sumo 10).

Se pide:

- Implementar la clase **Atención** con sus respectivos atributos, constructores y métodos para acceder y modificar sus atributos (respecto a los métodos de acceso sólo implemente los que vaya a utilizar). Además de todos los elementos necesarios para representar los diferentes tipos de atenciones.
- La veterinaria quiere conocer el promedio de ganancias entre todas sus atenciones. Implemente los métodos necesarios para calcular el promedio de ganancias de la veterinaria, teniendo en cuenta lo siguiente:
 - Para los **Tratamientos**, se calculan a partir del precio base. A ese valor se le suma un 5% si los síntomas son más de 3 y un 7.5% si son más de 6.
 - Para las **Intervenciones Quirúrgicas**, al igual que los tratamientos, se calculan a partir del precio base. En caso de que la intervención no haya sido programada, a ese valor inicial, se le suma un 15%. A su vez, si la intervención no fue programada, y fue durante alguno de estos feriados: 02/04/2025, 17/06/2025, 20/06/2025, 09/07/2025; se agrega un plus de 10.000 pesos. Por otro lado, si se utilizaron más de 5 medicamentos tiene un costo fijo extra de 20.000 pesos.
 - Para ambos casos, si la atención se realiza a un animal que no tiene el calendario de vacunación completo, se suma un 10% al costo total calculado de atención, para regularizar la situación y aplicar la/s vacuna/s correspondiente/s. Por otro lado, si el dueño paga en efectivo, la veterinaria ofrece un 12% de descuento sobre el total a pagar, en cambio, si paga con crédito, tiene un recargo de 5%. Con los otros medios de pagos, el valor permanece inalterable.

*Para ambos tipos de atenciones, si se cumplen las condiciones, los aumentos/descuentos son acumulables.

- Implementar los métodos necesarios para imprimir toda la información de la veterinaria y sus atenciones.
- Implementar un programa principal que instancie la veterinaria, agregue las atenciones y se evalúen todos los casos (no es necesario leer la información desde teclado). Después de la carga, hacer la impresión de los datos utilizando lo implementado en el inciso c) y, por último, informar la ganancia promedio.

Clases con MAYÚSCULA

Buenas Prácticas (Sube nota)

Clase Duenio

```
public class Duenio {
    private String nombre;
    private String apellido;
    private String dni;
    private String metodoPago; // "débito", "mercado pago", "crédito" o "efectivo"
```

atributos con minúscula

uso de Camel Case = mayúscula cuando empieza una nueva práctica

Constructor

```
public Duenio(String nombre, String apellido, String dni, String metodoPago) { // recibo datos por parámetro.
    this.nombre = nombre;
    this.apellido = apellido;
    this.dni = dni;
    this.metodoPago = metodoPago;
}
```

distingo atributo de clase

getters → obtener datos

```
public String getNombre() { return nombre; }
public String getApellido() { return apellido; }
public String getDni() { return dni; }
public String getMetodoPago() { return metodoPago; }
```

@Override

```
public String toString() { // básicamente concateno la info
    return "Dueño: " + nombre + " " + apellido + " (DNI: " + dni + ") | Pago: " + metodoPago;
}
```

→ devuelve un String

Clase Atención <<Abstract>> → Pq es EL molde

```
public abstract class Atencion {
    private int numero;
    private String fecha; // Formato "DD/MM/AAAA" → no se hagan lo pongan en String
    private String tipoAnimal; // Perro, gato
    private boolean vacunacionAlDia;
    private double precioBase;
    private Duenio duenio; → atributo q hace ref. al objeto "duenio"
```

los atributos está bueno ponerlos privados

```
public Atencion(int numero, String fecha, String tipoAnimal, boolean vacunacionAlDia, double precioBase, Duenio duenio) {
    this.numero = numero;
    this.fecha = fecha;
    this.tipoAnimal = tipoAnimal;
    this.vacunacionAlDia = vacunacionAlDia;
    this.precioBase = precioBase;
    this.duenio = duenio;
}
```

constructor

```
public int getNumero() { return numero; }
public String getFecha() { return fecha; }
public String getTipoAnimal() { return tipoAnimal; }
public boolean isVacunacionAlDia() { return vacunacionAlDia; }
public double getPrecioBase() { return precioBase; }
public Duenio getDuenio() { return duenio; }
```

// Método abstracto para el cálculo específico de cada tipo de atención

```
public abstract double calcularSubtotal(); → no tiene cuerpo qz Pq cada subclase lo va a implementar a su manera
```

// Método para calcular el monto final incorporando reglas generales de vacunación y pago

```
public double calcularMontoTotal() {
    double subtotal = calcularSubtotal();
    double porcentajeAumentoDescuento = 0.0; → inicializo
```

// Si no tiene vacunación completa, suma 10%

```
if (!vacunacionAlDia) {
    porcentajeAumentoDescuento += 0.10;
}
```

→ suma 10%

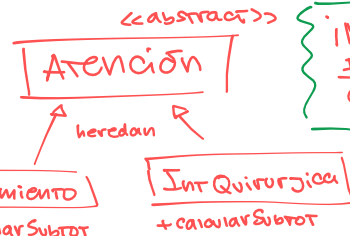
// Ajuste según método de pago

```
String pago = duenio.getMetodoPago().toLowerCase();
```

→ no hace falta

Pero lo convertí a minúscula

Porque desp comparo en minúscula



¡NUNCA
ESCRIBIR
CODIGO REPE!!
→ lo q sea = va
en el padre, no
en los hijos.

Cada una lo hace con
sus diferencias, pero
ambas calculan un
Subtotal.

```

if (pago.equals("efectivo")) {
    porcentajeAumentoDescuento -= 0.12; // 12% descuento
} else if (pago.equals("crédito") || pago.equals("credito")) {
    porcentajeAumentoDescuento += 0.05; // 5% recargo
}

```

```

return subtotal * (1.0 + porcentajeAumentoDescuento);
}

```

@Override → Pz sobrescribir el método toString para mostrar los datos de esta clase.

```

public String toString() {
    return "Atención N°: " + numero + " | Fecha: " + fecha + " | Animal: " + tipoAnimal +
        " | Vacunación: " + (vacunacionAlDia ? "Al día" : "Incompleta") +
        " | Precio Base: $" + precioBase + "\n " + dueño.toString();
}

```

Forma abreviada de ponerla. Ponerla o no. Poner solo V / F con boolean q ya tienen.

es una subclase q hereda de Atención.

```

public class Tratamiento extends Atención {
    private String descripcion;
    private int cantSintomas;

```

Constructor

```

public Tratamiento(int numero, String fecha, String tipoAnimal, boolean vacunacionAlDia, double precioBase, Duenio dueño,
String descripcion, int cantSintomas) {
    super(numero, fecha, tipoAnimal, vacunacionAlDia, precioBase, dueño);
    this.descripcion = descripcion;
    this.cantSintomas = cantSintomas;
}

```

llamo al constructor de la superclase Atención

```

public String getDescripcion() { return descripcion; }
public int getCantSintomas() { return cantSintomas; }

```

@Override

```

public double calcularSubtotal() {
    double subtotal = getPrecioBase();

```

Precio base heredado de Atención

```

if (cantSintomas > 6) {
    subtotal *= 1.075; // 7.5% de aumento si son más de 6 síntomas
} else if (cantSintomas > 3) {
    subtotal *= 1.05; // 5% de aumento si son más de 3 síntomas
}

```

→ aumenta este % → ¡MUCHOS SE EQUIVOCAN Y PONEN += !
 → sub *= 1,075 = sub += 0,075

```

return subtotal;
}

```

@Override

```

public String toString() {
    return super.toString() + "\n Tipo: Tratamiento | Desc: " + descripcion + " | Síntomas: " + cantSintomas;
}

```

Para reutilizar la cadena de Atención → queda el toString de + lo q se agregue (los concatenas)

Clase Intervención quirúrgica

```

public class IntervencionQuirurgica extends Atención {
    private int duracionMinutos;
    private boolean esProgramada;
    private String[] medicamentos;
    private int cantMedicamentos;

```

→ nombres med.
 → dimensión lógica, consigna clara máx 10

Constructor

```

public IntervencionQuirurgica(int numero, String fecha, String tipoAnimal, boolean vacunacionAlDia, double precioBase,
Duenio dueño, int duracionMinutos, boolean esProgramada) {
    super(numero, fecha, tipoAnimal, vacunacionAlDia, precioBase, dueño);
    this.duracionMinutos = duracionMinutos;
    this.esProgramada = esProgramada;
    this.medicamentos = new String[10]; // A lo sumo 10
    this.cantMedicamentos = 0;
}

```

Por ahora no cargue ninguno.

getters }

```
public double calcularPromedioGanancias() {
    if (dimL == 0) return 0.0; // caso base = no se carga ninguna arehcion → no se p. nunca lo ponen
    double sumaTotal = 0.0; // acumulador // es el caso base, ya tiene q
    for (int i = 0; i < dimL; i++) { // recorro vector hasta dimL estar en el molde.
```

```

        sumaTotal += atenciones[i].calcularMontoTotal();
    }
    return sumaTotal / dimL;
}

```

vector *Consigo daro usando POLIMORFISMO*

6 PROMEDIO tmb se equivcan banda en esto y no se Pg.

```

public void imprimirInformacion() {
    System.out.println("===== REGISTRO DE ATENCIONES VETERINARIAS 2025 =====");
    System.out.println("Total de Atenciones Registradas: " + dimL);
    System.out.println("-----");
    for (int i = 0; i < dimL; i++) { // bucle pasa por cada atencion
        System.out.println("Atención #" + (i + 1) + ":");
        System.out.println(atenciones[i].toString());
        System.out.println("Monto Total Facturado: $" + String.format("%.2f", atenciones[i].calcularMontoTotal()));
        System.out.println("-----");
    }
}

```

el # esta de deco *tmb de deco.*

Clase ProgramaPrincipal

```

public class ProgramaPrincipal {
    public static void main(String[] args) {
        Veterinaria v = new Veterinaria(); // instancio objeto de la clase Vet

        // Dueños de prueba Recen q no les pida leer de teclado
        Duenio d1 = new Duenio("Juan", "Pérez", "38123456", "efectivo");
        Duenio d2 = new Duenio("María", "Gómez", "40987654", "crédito");
        Duenio d3 = new Duenio("Carlos", "López", "35111222", "mercado pago");

        // Caso 1: Tratamiento (>6 síntomas, sin vacuna al día, pago en efectivo)
        Tratamiento t1 = new Tratamiento(101, "15/03/2025", "Perro", false, 25000.0, d1, "Infección respiratoria severa", 7);
        v.agregarAtencion(t1); → guarda el objeto Tratamiento en el vector de la Vet.

        // Caso 2: Intervención Quirúrgica (no programada, fecha feriado 25/05/2025 o 09/07/2025, >5 medicamentos, vacuna al día, crédito)
        IntervencionQuirurgica i1 = new IntervencionQuirurgica(102, "09/07/2025", "Gato", true, 80000.0, d2, 90, false);
        for (int k = 1; k <= 6; k++) { // cargo 6 meds para probar.
            i1.agregarMedicamento("Medicamento " + k); → agrego med. en vect. de meds.
        }
        v.agregarAtencion(i1); → guardo obj. en vect.

        // Caso 3: Tratamiento (3 síntomas o menos, vacuna al día, mercado pago)
        Tratamiento t2 = new Tratamiento(103, "10/08/2025", "Loro", true, 15000.0, d3, "Control de rutina y plumaje", 2);
        v.agregarAtencion(t2); → guardo obj. en vect.

        // Imprimir todos los datos de la veterinaria y sus atenciones
        v.imprimirInformacion(); método de Vet

        // Informar el promedio de ganancias
        System.out.println("Ganancia Promedio por Atención: $" + String.format("%.2f", v.calcularPromedioGanancias()));
    }
}

```

muestra en consola *no need*